

SDLC & STLC. Software Development Methodologies

Методології розробки програмного забезпечення (SDLC & STLC)

Автор: Анна Гуртова (Anna Hurtova)

Курс: Beetroot Academy / Helvetas Ukraine ("Стійкість")

Роль: Trainee QA Manual Engineer

Дата: Червень 2026

Блок 1: Порівняльна таблиця методологій розробки ПЗ (SDLC & STLC)

№	МЕТОДОЛОГІЯ	СИЛЬНІ СТОРОНИ	СЛАБКІ СТОРОНИ	ГАЛУЗЬ ЗАСТОСУВАННЯ	КОЛИ ВИКОРИСТОВУВАТИ
1	Waterfall (Каскадна)	Чітка прогнозованість: фіксація бюджету, термінів та обсягу робіт ще до старту написання коду. Стабільність вимог: вимоги не змінюються в процесі, що мінімізує ризик технічного хаосу. Висока прозорість: замовник завчасно бачить, скільки часу займатиме кожен крок. Документація пишеться завчасно — знижується ризик непорозуміння.	Катастрофічна ціна помилки: якщо QA знаходить баг в архітектурі під час фінального тестування, доведеться переписувати весь готовий продукт. Жорсткість: нові вимоги клієнта посеред шляху неможливо прийняти. Пізній фідбек: клієнт бачить готовий продукт лише наприкінці.	Критично важливі системи (Mission Critical): авіаційна та космічна промисловість, медичне обладнання, банківські системи, державні реєстри. Обґрунтування: помилка коштуватиме людських життів або катастрофічних фінансових втрат.	На проєктах з високою формалізацією процесів, де робота ведеться суто по регламенту, а опис проєкту сформульовано точно та зафіксовано вимоги.
2	V-Model (V-подібна)	QA-орієнтованість: тестування починається одночасно з аналізом вимог. Мінімізація ризиків: кожен етап розробки (SDLC) має свій дзеркальний етап перевірки (STLC). Висока якість документації: детальний аналіз ризиків знижує вірогідність критичних помилок.	Відсутність гнучкості: при будь-яких змінах вимог необхідно повертатись на початок процесу. Паперове перевантаження: вимагає створення величезної кількості проміжних звітів та тестової документації на кожному мікроетапі.	Медична апаратура, автомобільні системи керування (наприклад, безпілотники). Обґрунтування: кожен технічний крок має бути миттєво підтверджений суворим тестам на відповідність промисловим стандартам.	На проєктах, де потребується масштабне тестування, суворі правила в роботі та чіткі фіксовані вимоги.
3	Iterative (Ітераційна)	Еволюційний підхід: продукт створюється одразу весь, але в дуже простій версії, а потім з кожною ітерацією обростає деталями. Швидке виявлення помилок: користувачі отримують працюючий продукт на ранній стадії.	Ризик «безкінечного» покращення: складно вчасно зупинитися, якщо немає чітких критеріїв завершення. Складність для архітектури коду через неясність вимог на початку.	Текстові нейромережі, рекомендаційні системи музичних/відеоплатформ, пошукові системи. Обґрунтування: помилка не є критичною для життя; з кожною ітерацією алгоритм доводиться до ідеалу.	У проєктах, де вимоги ще формуються, де потрібен оперативний запуск, або у великих проєктах з обмеженими ресурсами.
4	Incremental (Інкрементна)	Швидкий випуск робочого коду: продукт розбивається на повноцінні функціональні елементи (інкременти),	Високі ризики на етапі інтеграції: новий інкремент під час злиття коду може зламати вже працюючі частини. Це	Великі комерційні платформи, маркетплейси, стрімінгові сервіси, корпоративні CRM-системи.	Коли потрібен реальний фідбек від живих користувачів на перші базові функції, а загальні

№	МЕТОДОЛОГІЯ	СИЛЬНІ СТОРОНИ	СЛАБКІ СТОРОНИ	ГАЛУЗЬ ЗАСТОСУВАННЯ	КОЛИ ВИКОРИСТОВУВАТИ
		кожен з яких є протестованим та готовим до релізу.	вимагатиме від тестувальника постійного масштабного регресійного тестування. Складність проектування початкової архітектури: якщо неправильно спроектувати каркас БД — додавати нові блоки буде неможливо без глобального переписування.	<i>Обґрунтування:</i> бізнес не чекатиме рік — запускають частинами, де кожна одразу приносить комерційну користь.	вимоги до продукту визначені на старті.
5	Spiral (Спіральна)	Ультимативний менеджмент ризиків: кожен виток спіралі починається з прискіпного прорахунку фінансових, технічних та юридичних ризиків, що мінімізує ймовірність провалу.	Висока вартість та складність управління: потребує постійного залучення високооплачуваних аналітиків для оцінки ризиків на кожному колі; проєкт може зациклитись на етапі планування.	Масштабні інноваційні проєкти, військово-оборонний сектор, розробка ігор-блокбастерів. <i>Обґрунтування:</i> коли проєкт вимірюється сотнями мільйонів — компанія не може діяти навмання.	Коли проєкт технічно унікальний, є великий бюджет, а вимоги складні чи розмиті на старті і потребують постійної верифікації через прототипи.
6	Agile / Scrum (Ітеративно-інкрементна)	Максимальна гнучкість: команда може миттєво змінювати пріоритети на початку кожного спринту під поточні запити ринку. Швидка окупність: замовник отримує працюючий протестований функціонал кожні 2 тижні.	Розмиті фінальні межі проєкту: через постійні зміни вимог дуже важко спрогнозувати кінцеві терміни релізу та фінальну вартість продукту.	Комерційні та веб/мобільні застосунки, стартапи, E-commerce. <i>Обґрунтування:</i> ринок і технології змінюються щодня — продукт має постійно еволюціонувати.	Коли компанії потрібно терміново вийти на ринок з базовою версією і далі постійно покращувати її на основі реальних відгуків.
7	Kanban (Agile Kanban)	Максимальна гнучкість без жорстких рамок: немає фіксованих спринтів, пріоритети можна змінювати будь-коли. Робота йде безперервним рівномірним потоком, що виключає стреси «кінця спринту».	Складність довгострокового планування: через безперервність потоку важко назвати точну дату готовності. Якщо команда не має високої самодисципліни, проєкт може розтягнутися в часі.	Проєкти техпідтримки, сервісні служби, розвиток уже діючих продуктів. <i>Обґрунтування:</i> завдання приходять непередбачувано; Kanban дозволяє миттєво взяти критичну помилку в роботу, не чекаючи початку нового спринту.	Коли продукт уже випущений (етап підтримки SDLC), коли вимоги змінюються щодня, або коли команда невелика і фокус на швидкості вирішення поточних завдань.

Блок 2: Чому з'явився Agile-маніфест? Які проблеми він вирішує і чи є це успіхом?

Передумови появи Agile-маніфесту

Agile-маніфест, сформульований у **2001 році** групою провідних інженерів, з'явився не просто як нова модна теорія, а як вимушена, закономірна реакція на глибоку системну кризу в IT-індустрії (технології розвивалися стрімко, а методи керування проєктами лишались застарілими). До цього моменту розробка програмного забезпечення повністю копіювала процеси важкої промисловості та будівництва, використовуючи каскадну модель (Waterfall).

Головними причинами появи Маніфесту стали:

- **Катастрофічна негнучкість процесів:** традиційний підхід вимагав детального опису 100% вимог на роки наперед у гігантських паперових ТЗ. На момент, коли розробники нарешті писали код за дворічним ТЗ, ринок і потреби клієнта вже повністю змінювалися. Продукт виходив застарілим і нікому не потрібним.
- **Криза комунікації та «паперовий хаос»:** процеси були заблоковані бюрократією. Команди витрачали місяці на погодження звітів замість того, щоб створювати працюючий код. Взаємодія з клієнтом нагадувала суворі судові суперечки.
- **Економічна неефективність та пізнє тестування:** клієнти бачили готову систему лише на фінальній стадії. Якщо на етапі тестування виявлялася помилка в архітектурі — проєкт або закривався з мільйонними збитками, або потребував колосальних коштів на переробку.

Першопричина: критична потреба ІТ-бізнесу змінити пріоритети — відійти від жорсткої паперової бюрократії у бік швидкості, адаптивності та реальної цінності для користувача.

Які проблеми вирішує Agile?

Agile-маніфест запропонував абсолютно нову систему координат, що базується на **4 цінностях** та **12 принципах**. Він успішно вирішує кілька фундаментальних проблем:

- **Проблема швидкості виходу на ринок (Time-to-Market):** завдяки розбиттю на короткі ітерації, клієнт отримує перші працюючі функції вже за кілька тижнів, а не через роки. Продукт починає окупатися відразу.
- **Проблема невизначеності вимог:** Agile легалізував зміни. Якщо у конкурентів з'явилася нова опція — команда може миттєво змінити пріоритети на початку наступного спринту. Зміна вимог більше не вважається катастрофою.
- **Проблема ізоляції QA та розробників:** тестування стало безперервним процесом всередині кожної ітерації. Помилки знаходяться і виправляються миттєво, що суттєво знижує вартість їх усунення.

Чи є це успіхом?

Так, Agile — це беззаперечний еволюційний успіх, який назавжди змінив світову бізнес-культуру. Сьогодні за його принципами працюють не лише ІТ-гіганти (**Apple, Spotify, Amazon**), а й банківські структури, маркетинг та логістика.

Проте, цей успіх має свої нюанси:

- Agile є успішним лише там, де вимоги динамічні (мобільні застосунки, веб-сайти, стартапи). Він не підходить для будівництва космічних ракет, авіації чи медицини, де каскадні моделі з жорсткою документацією залишаються єдиним безпечним вибором.
- Багато компаній декларують, що працюють за Agile, купують дошки Канбан, проводять мітинги, але всередині залишаються жорсткою бюрократичною машиною. *Успіх гнучкої методології залежить не від інструментів, а від зміни мислення усієї команди.*

Блок 3: Обґрунтування вибору Agile (Scrum) для стартапу

Для стартапу з обміну світлинами котиків обрано гнучку методологію **Agile**, а саме її конкретний фреймворк — **Scrum**.

Чому Scrum, а не Kanban?

На етапі запуску нового продукту критично важливо рухатися фіксованими часовими відрізками (спринтами) та мати чіткий дедлайн для створення першої працюючої версії. Scrum фокусує команду на швидкому результаті до кінця кожного спринту, тоді як Kanban більше орієнтований на безперервний потік завдань без чітких дедлайнів — це більше підходить для етапу підтримки вже готового додатку, а не для його агресивного старту на ринку.

Що дає Agile/Scrum для мобільного стартапу?

- **Економія бюджету та швидкий запуск:** першою метою буде за пару місяців випустити найпростішу робочу версію (MVP) з базовим функціоналом. Побачивши реакцію реальних користувачів, можна буде інвестувати в більш складний функціонал і поступово масштабувати проєкт.

- **Синхронізація SDLC та STLC:** перевірка логіки майбутнього функціоналу ще на етапі ідеї. Це дозволить виявляти логічні помилки в архітектурі та ТЗ, коли їх виправлення коштуватиме найдешевше.
- **Гарантія стабільності в кожному релізі:** тестування здійснюватиметься паралельно з розробкою в кожному спринті.
- **Адаптивність до ринкових змін:** відслідковуючи поведінку цільової аудиторії та їх запити (наприклад, інтеграція з B2B-сегментом — ветклініками, виробниками кормів), Agile дозволить миттєво переприоритизувати беклог, змінити вектор розвитку в наступному спринті, не втрачаючи ресурси на переписування застарілих ТЗ.

Висновок: Agile (Scrum) забезпечить оптимальний баланс між швидкістю виходу на ринок, раціональним використанням бюджету та жорстким контролем якості продукту на кожному етапі його життєвого циклу.